

10.How to install Django and create project



PHAN DY

MIT, Build Bright University

Tell: 097 239 3060

Email: phandy010@gmail.com

How to install Django and create project

Step 1 - Open Powershell

Firstly, you need to open Powershell. There are two ways in which you can do that. Either search for Powershell in the Windows search box or open the Run dialog box with (WIN+R). Once the dialog box is open, type Powershell and click **OK**.

Once the Powershell window is open, you will verify Python installation in the next section.

Step 2 - Verify Python Installation

Before installing Django, you need to make sure that Python is installed on your system.

Type the following command to verify the installation:

- *python -V*

You will get the below output:

Output

```
PS C:\Users\Username> python -V
Python 3.9.7
```

At the time of writing this article, I have Python version 3.9.7 and yours may be different, but the output will confirm if you have Python installed or not.

Step 3 - Upgrade Pip

Pip comes by default with Python but generally with an old version. Use the following command to upgrade pip:

- *python -m pip install --upgrade pip*

Step 4 - Create Project Directory

Now, create a project directory in which you will keep the Django repository. Name the project based on your choice.

1. cd into your Desktop directory:
 - *cd Desktop*
2. Use the following command to create a directory:
 - *mkdir demo_project*
3. Now, enter the newly-created directory using the below command:
 - *cd demo_project*

Step 5 - Create a Virtual Environment

In this step, you will create a virtual environment for you project. With the help of a virtual environment, you can create an isolated environment that won't affect other Python projects.

If you are not using a virtual environment, all your projects will be using the same Django version installed globally. It can lead your projects to fail when there is an update in Django with breaking changes.

Use the following command to create a virtual environment:

- *python -m venv venv*

The above command will create a directory called venv inside your project directory.

Confirm if the venv directory has been created using the below command:

- *ls*

Step 6 - Activate the Virtual Environment

Now, you will activate the virtual environment in your directory.

Run the following command:

- *venv\Scripts\activate*

Once, it is activated you will see venv at the beginning:

- *(venv) PS C:\Users\Stanley\Desktop\django_project>*

Step 7 - Install Django on Windows

1) You will be installing Django using pip, execute the following command:

- *pip install django*

The above command will install the latest version of Django.

2) In case you want to install a different Django version, mention the version as follows:

- *pip install django==3.1*

3) Now, verify if the installation has been done successfully:

- *django-admin --version*

4) You will get an output like below:

Output

```
PS C:\users\stanley\Desktop\django_project> django-admin --version
3.9.7
```

Step 8 - Create a Django Project

1) You can create your first Django project using the following command:

- *django-admin startproject test_project*

2) Move to the *test_prohect*:

- *cd test_project*

3) Type the following command to see the contents of the project directory:

- *ls test_project*

Step 9 - Run the Development Server

1) Once the project is created, start the Django development server:

- *python manage.py runserver*

2) Now, you can access your project at <http://127.0.0.1:8000/>.

3) If you want to stop the server, press CTRL+C and to deactivate the virtual

- *environment type deactivate.*

Step 10 – Create App

- *python .\manage.py startapp APP*

Step 11 – Migrate table

- *python ./manage.py migrate*
- *python manage.py makemigrations*

Step 12 – Create SupperUser

- *python ./manage.py createsuperuser*

Step 13 – Create Model

1. Don't need to add id (it is generate by it self)

Step 14 – Configure App

```
STATICFILES_DIRS = [  
    os.path.join(BASE_DIR, 'static'),  
]  
  
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'APP',  
]  
]
```

Step 14 – Use mysqlclient

```
'default': {  
    'ENGINE': 'django.db.backends.mysql',  
    'NAME': 'demo',  
    'USER': 'root',  
    'PASSWORD': '',  
    'HOST': 'localhost',  
    'PORT': '3306',  
}
```

Demo Project Python Django MySQL CRUD

In this [Python Django Tutorial](#), I will show you Python Django MySQL **CRUD** operations, where you will understand how to implement CRUD functionality in the Django project.

Additionally, I will explain to you how you can set up your Django project using a custom database such as MySQL.

Finally, you will learn about CRUD operations by building a small project which is based on the blog. Where the user can create a new blog or view the blog, modify the existing one and delete the specific blog.

Python Django MySql CRUD

Django is a web framework where you can implement CRUD operations and is based on the MVT (Model View Template) architecture. **CRUD** stands for **Create**, **Retrieve**, **Update**, and **Delete**, and these are the fundamental operations that any web developer should be familiar with.

What is the importance of CRUD operation?

CRUD operation provides the required tools to manipulate data in the system that allows user to do the following things according to the requirement:

- **Create:** In a database table, create or add new entries.
- **Retrieve:** Read, Retrieve or Fetch all or some entries from the table in a database.
- **Update:** In a database table, update or amend existing entries.
- **Delete:** In a database table, delete existing records.

Without the CRUD operation, a system or website becomes static or non-interactive, so when CRUD operation is implemented, it makes the website dynamic and user-friendly.

- When CRUD operations are implemented in an optimized way, then it retrieves or manipulates the data in a more efficient way that improves the system or website performance.
- CRUD operation provides integrity when it is implemented in a proper manner that makes sure user data remains consistent.

Now you know what CRUD operations are and their importance. Let's learn about CRUD operation in Django by building a blog project.

Python Django MySQL CRUD Project Setup

In this section, you will learn how to set up a Django project to perform CRUD operations with MySQL.

CREATE VIRTUAL ENVIRONMENT:

First, create a virtual environment named '**env**' using the below command.

- ***python -m venv env***

Activate the environment '**env**' using the below code.

- ***env\Scripts\activate***

Install the latest version of Django using the below command.

- ***pip install django***

CREATE PROJECT:

Then create a Django project named '**blog_project**'. For this, type the following command in the terminal.

- ***django-admin startproject blog_project***

Change the directory to the **blog_project**.

- ***cd blog_project***

CREATE APP:

Next, within the project, we will create an app named **blog_app**. And later on this, we'll perform CRUD operations. Type the following command in the terminal to create an app.

- ***python manage.py startapp blog_app***

```
Microsoft Windows [Version 10.0.22621.2283]
(c) Microsoft Corporation. All rights reserved.

D:\Django\django_crud>python -m venv env
D:\Django\django_crud>env\Scripts\activate
(env) D:\Django\django_crud>pip install django
Collecting django
  Using cached Django-4.2.5-py3-none-any.whl (8.0 MB)
Collecting sqlparse>=0.3.1
  Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
Collecting tzdata
  Using cached tzdata-2023.3-py2.py3-none-any.whl (341 kB)
Collecting asgiref<4,>=3.6.0
  Using cached asgiref-3.7.2-py3-none-any.whl (24 kB)
Collecting typing-extensions>=4
  Using cached typing_extensions-4.8.0-py3-none-any.whl (31 kB)
Installing collected packages: tzdata, typing-extensions, sqlparse, asgiref, django
Successfully installed asgiref-3.7.2 django-4.2.5 sqlparse-0.4.4 typing-extensions-4.8.0 tzdata-2023.3

[notice] A new release of pip is available: 23.0.1 -> 23.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(env) D:\Django\django_crud>django-admin startproject blog_project
(env) D:\Django\django_crud>cd blog_project
(env) D:\Django\django_crud\blog_project>python manage.py startapp blog_app
(env) D:\Django\django_crud\blog_project>
```

CREATE TEMPLATES:

After that, create the **templates** folder in the Django app '**blog_app**', in that folder create the following empty template that will be used later.

- **search.html**: To display the fetched data.
- **create.html**: To add a new blog.
- **update.html**: To update the existing blog.
- **remove.html**: To delete a blog entity.

✓ BLOG_PROJECT

✓  blog_app

>  migrations

✓  templates

 create.html

 remove.html

 search.html

 update.html

 __init__.py

 admin.py

 apps.py

 models.py

 tests.py

 views.py

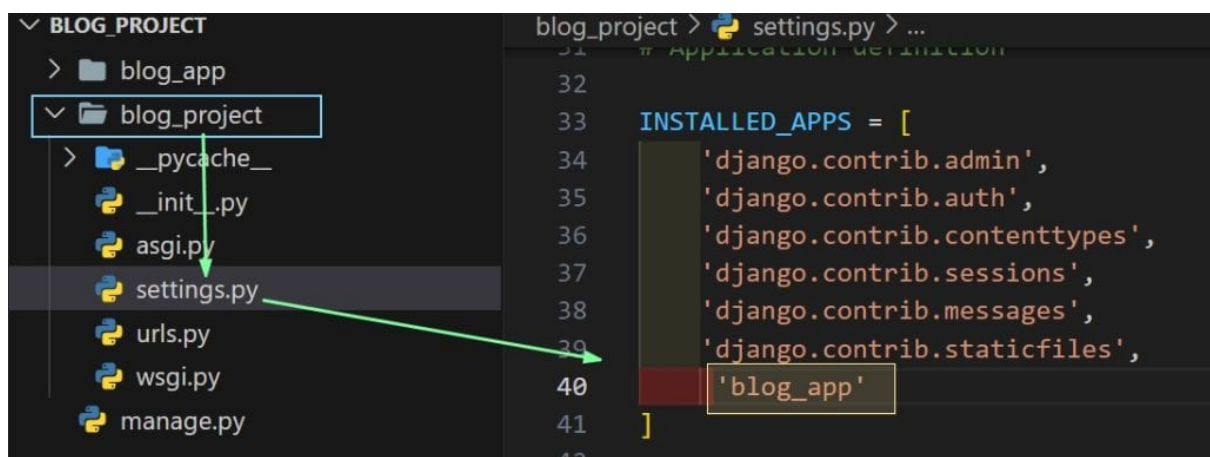
>  blog_project

 manage.py

INSTALL APP:

Then, in the **settings.py** file of your Django project '**blog_project**' include the '**blog_app**' under the section **INSTALLED_APPS**.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'blog_app'  
]
```

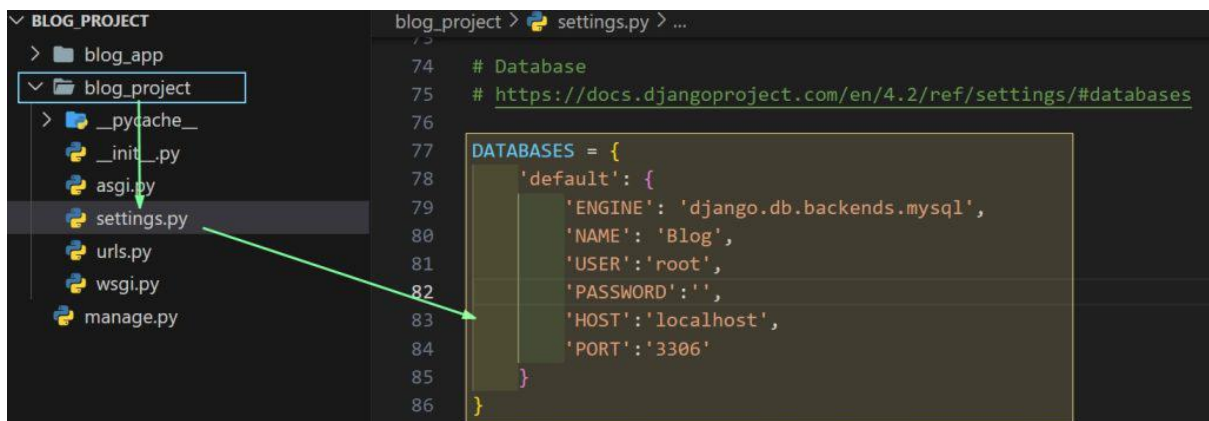


DATABASE SETUP:

After this, set up the database, in this blog project you will use the MySQL database. To set up the MySQL database add the following database configuration in the setting.py file of your Django project '**blog_project**'.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',
```

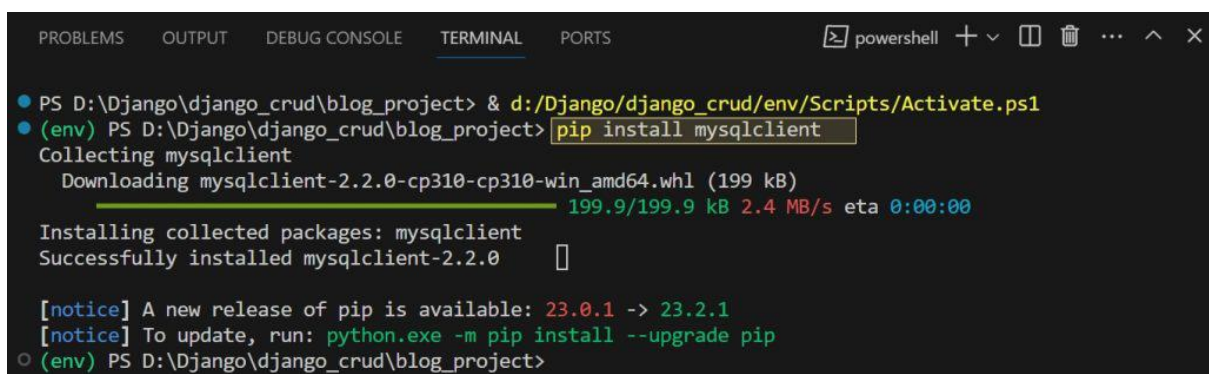
```
'NAME': 'Blog',
'USER': 'root',
'PASSWORD': '',
'HOST': 'localhost',
'PORT': '3306'
}
}
```



Install MySQL client:

Then open the terminal and install the MySQL client for Python.

- ***pip install mysqlclient***



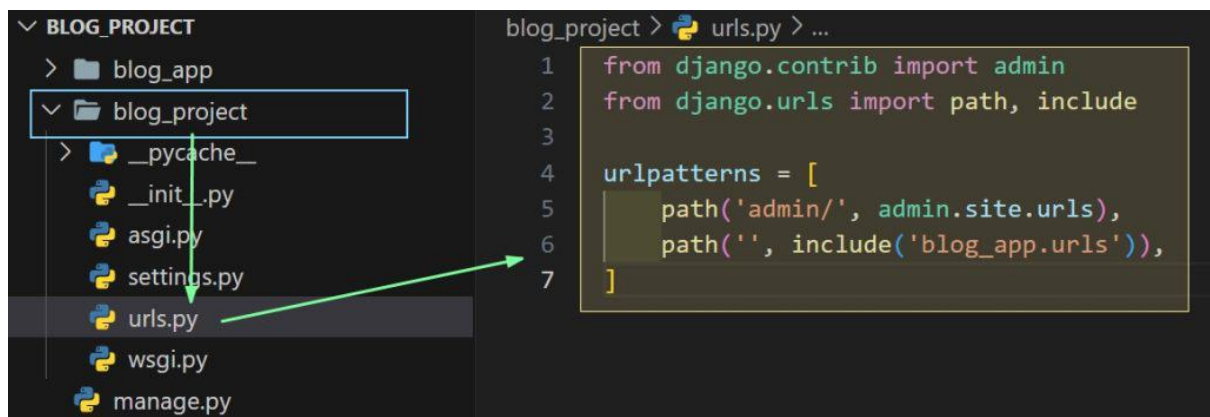
Creating URL Patterns Python Django MySQL CRUD

Create URL patterns at project and app levels, so follow the below steps:

PROJECT URL:

Open the `urls.py` file of your Django project '`blog_project`' and add the following lines of code.

```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('blog_app.urls')),
]
```

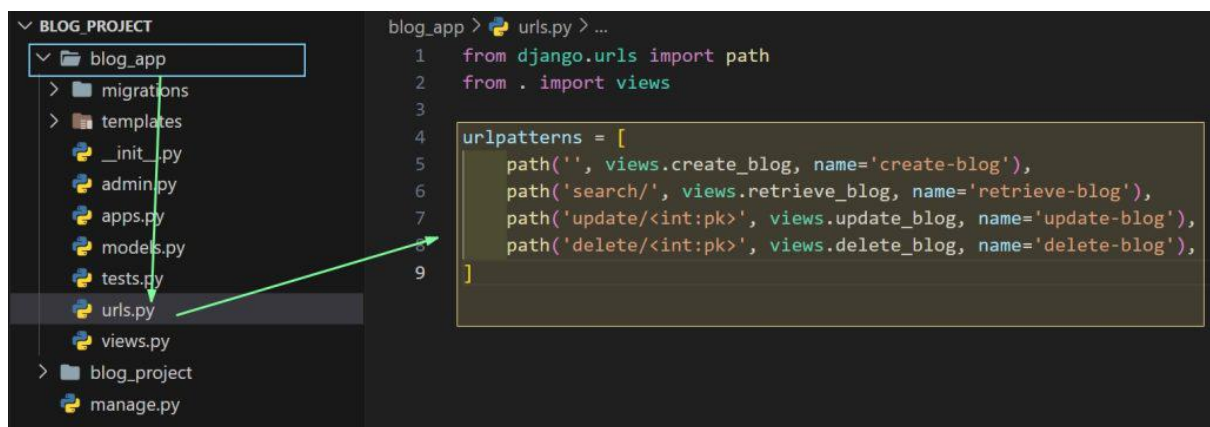


APP URL:

Next, create a new file called `urls.py` in your Django app `'blog_app'` and add the following lines of code.

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.create_blog, name='create-blog'),
    path('search/', views.retrieve_blog, name='retrieve-blog'),
    path('update/<int:pk>', views.update_blog, name='update-blog'),
    path('delete/<int:pk>', views.delete_blog, name='delete-blog'),
]
```



In the above code, each URL pattern corresponds to a specific view such as URL path `'search/'` will call the view `retrieve_blog` and so on. In the upcoming section, you will create these views.

Python Django MySQL CRUD Create the Model

Create the model named 'Blog' which represents the database schema or table named 'Blog'.

CREATE MODEL:

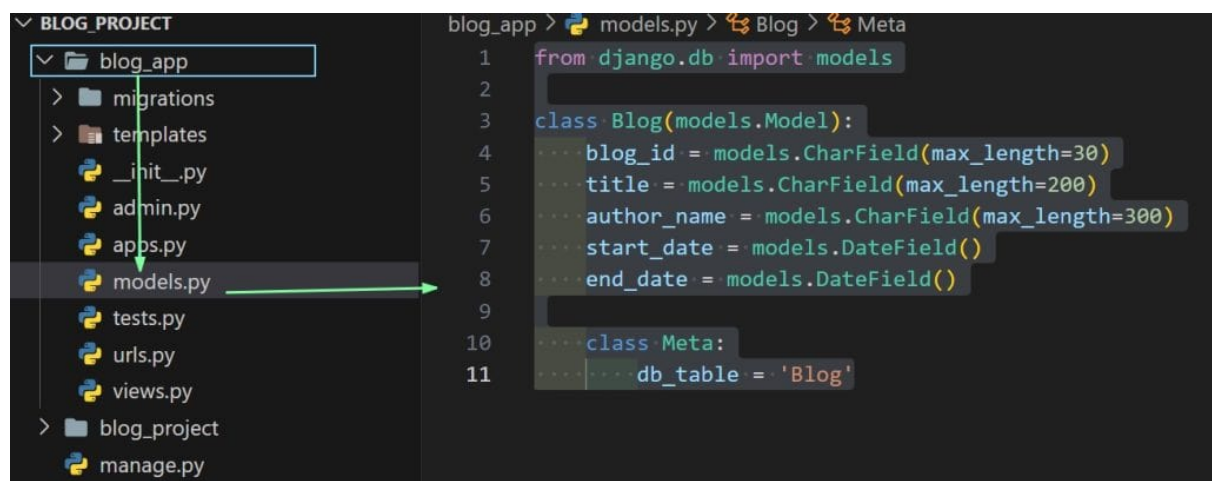
So, to create a model 'Blog' open the **models.py** file of your Django app 'blog_app' and add the following lines of code.

```
from django.db import models

class Blog(models.Model):

    blog_id = models.CharField(max_length=30)
    title = models.CharField(max_length=200)
    author_name = models.CharField(max_length=300)
    start_date = models.DateField()
    end_date = models.DateField()

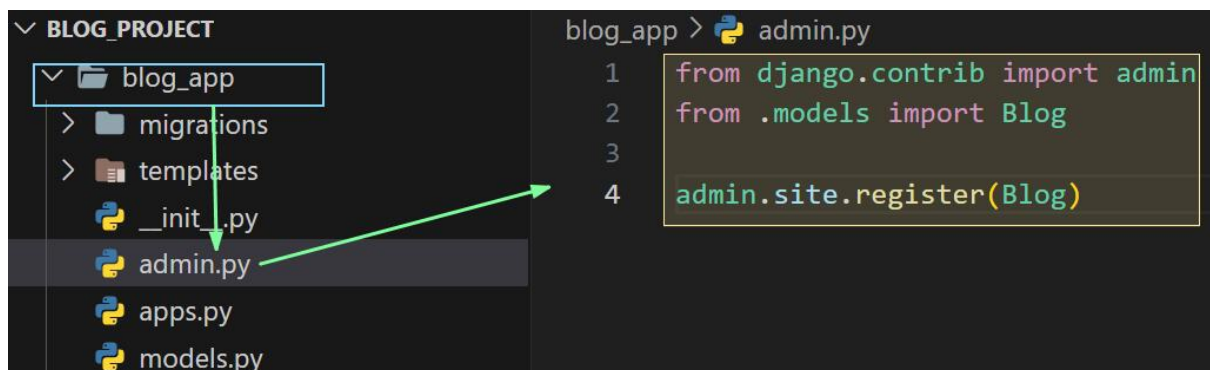
    class Meta: db_table = 'Blog'
```



In the above code, line 3 defines the model **Blog** with field lists **blog_id**, **title**, **author_name** of field class type (data type) **CharField** and the **start_date**, **end_date** of field class type **DateField**.

After defining the model, register the model in the **admin.py** file of your Django app 'blog_app' by adding the following lines of code.

```
from django.contrib import admin
from .models import Blog
admin.site.register(Blog)
```



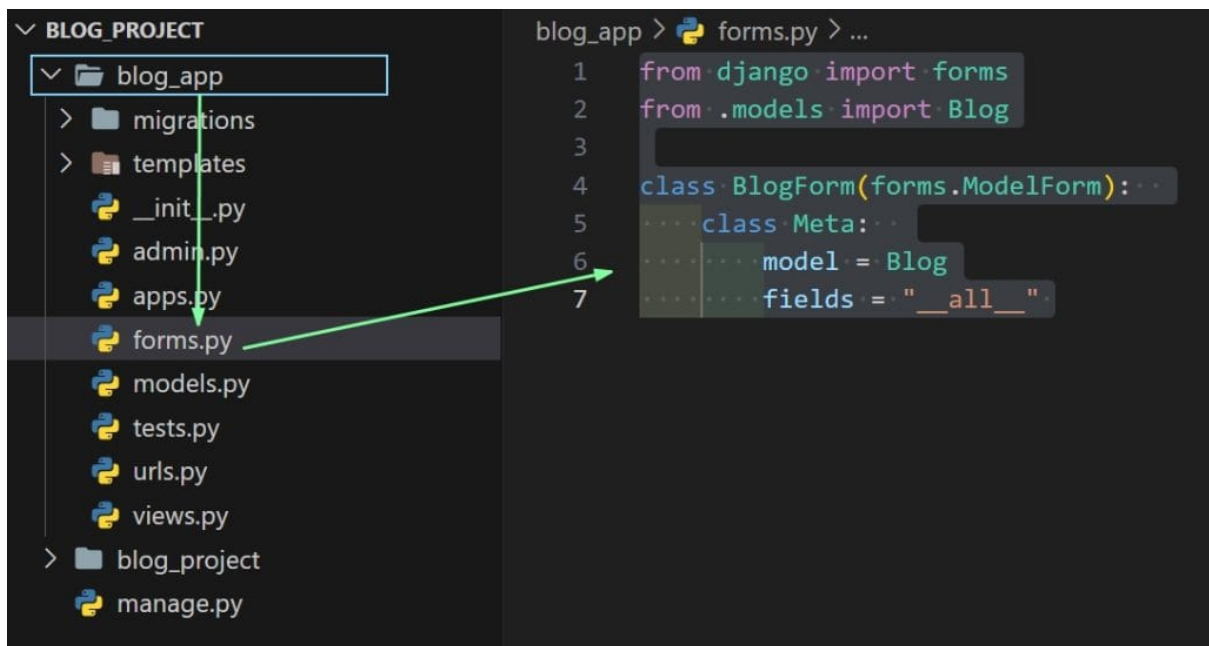
In the above code at line 4 register the model Blog to appear on the Admin panel.

Python Django MySQL CRUD Creating Form

After creating the model, create a Django form based on the model 'Blog', this form will appear on the template.

CREATE FORM: Create a new file called **forms.py** in your Django app 'blog_app' and in that file add the following lines of code.

```
from django import forms
from .models import Blog
class BlogForm(forms.ModelForm):
    class Meta:
        model = Blog
        fields = "__all__"
```

In the above code from lines 1 to 2 import the **forms** from the django and the model **Blog** from the current directory **models.py** file.

Then create the class **BlogForm** which inherits the **ModelForm**, the **ModelForm** takes the model and fields and automatically creates a form based on the provided model which is **Blog** in this case.

If you want to know more about **ModelForm**, then refer to the tutorial [Create Model Form in Django](#)

Creating View Functions Python Django MySQL CRUD

If you remember you have specified the different views in the **urls.py** file of your Django app **'blog_app'**. Let's create that view one by one, to create a view open the **views.py** file of your Django app and add the following views.

CREATE VIEW: Add the following code in the **views.py** file.

```
from django.shortcuts import render, redirect
from .forms import BlogForm

def create_blog(request):
    if request.method == "POST":
        form = BlogForm(request.POST)
        if form.is_valid():
            try:
                form.save()
                return redirect('search/')
            except:
                pass
    else:
        form = BlogForm()
    return render(request, 'create.html', {'form':form})
```

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'BLOG_PROJECT' with a sub-project 'blog_app'. Inside 'blog_app', there are files like 'migrations', 'templates', 'urls.py', and 'views.py'. A green arrow points from 'views.py' in the file explorer to the code editor. The code editor shows the following Python code:

```
1 from django.shortcuts import render, redirect
2 from .forms import BlogForm
3
4
5 def create_blog(request):
6     if request.method == "POST":
7         form = BlogForm(request.POST)
8         if form.is_valid():
9             try:
10                 form.save()
11                 return redirect('search/')
12             except:
13                 pass
14         else:
15             form = BlogForm()
16         return render(request, 'create.html', {'form': form})
17
18
```

In the above code, line 5 defines the view `create_blog` that accepts the parameter request object.

Then line 6, checks if the incoming request is a **POST** method or not. If the incoming request is the POST, then it creates a form instance using the **BlogForm** by passing the (**request.POST which represents the submitted form data**) and the files.

After this line 8 checks if the form is valid or not by calling the method `is_valid()` on the **form** instance. if the form is valid, it does the following thing:

- First, it saves the form in the database by calling the **save()** method on the **form**, after it redirects the user to the page **'search/'**.

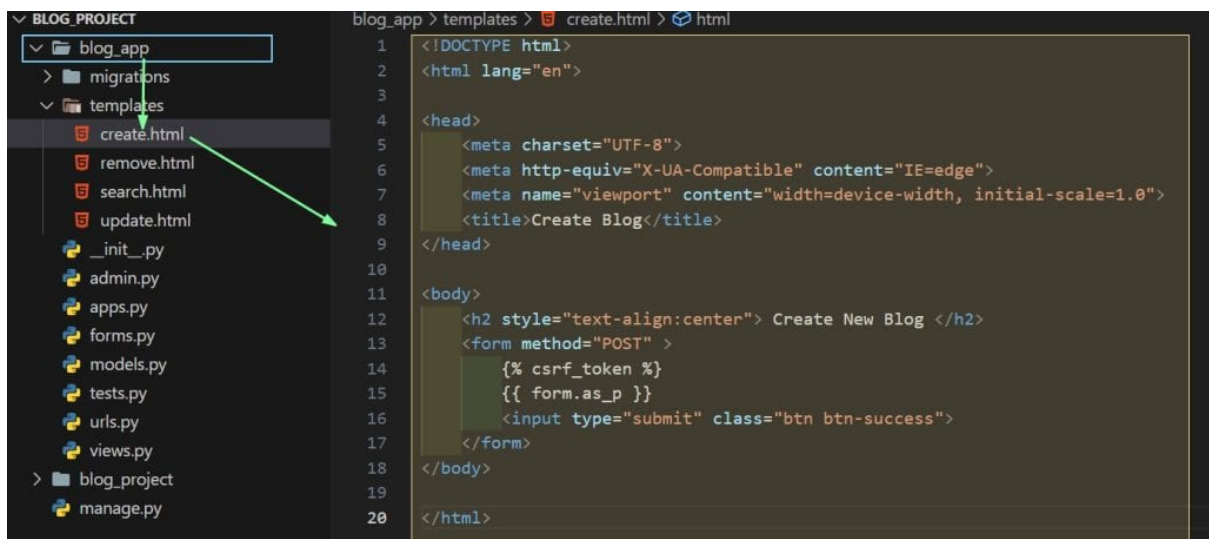
Then at line 14 in the else part, if the incoming request is not POST, then an empty form is initialized using the **form = BlogForm()**. At line 16, render the template **'create.html'** with context which contains the empty form.

Now add the following lines of code in the create.html file that you have created in the templates folder of your Django app **'blog_app'**.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Create Blog</title>
</head>
<body>
  <h2 style="text-align:center"> Create New Blog </h2>
  <form method="POST" >
    {% csrf_token %}
    {{ form.as_p }}
    <input type="submit" class="btn btn-success">
  </form>
</body>
</html>

```



In the above code,

- Here, we use the **form** element to create an HTML form for user input.

- We also add **method=POST** to the form tag.
- Then, we use `{% csrf_token %}` to protect our form from cyber-attacks.
- Next, we render Django forms as paragraphs which will look like this.

← → ↻ ⓘ 127.0.0.1:8000

Create New Blog

BlogId:

Title:

Author Name:

Start Date:

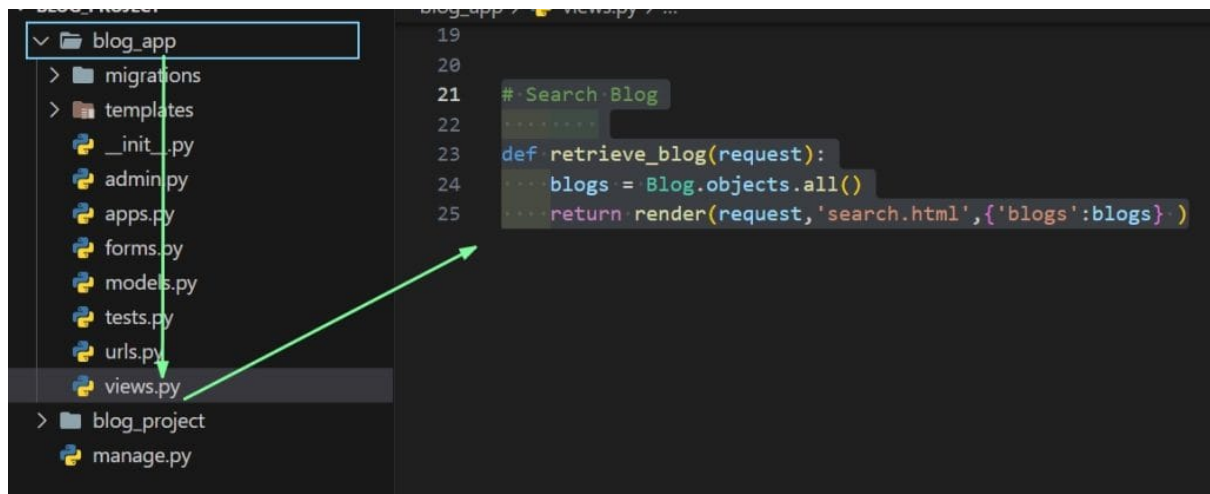
End Date:

Then create a view that will return all the posts from the database and pass those post to the template.

RETERIVE VIEW:

Add the following code in the **views.py** file.

```
from django.shortcuts import render, redirect
from .models import Blog
# Search Blog
def retrieve_blog(request):
    blogs = Blog.objects.all()
    return render(request, 'search.html', {'blogs':blogs} )
```



In the above code, line 23 defines the view function **retrieve_blog** that takes the request parameter in the form of an object.

Then the code at line 24 **Blog.objects.all()** returns all the blogs from the database as objects or query sets. After this, at line 25 render the template **search.html** with context that contains all the blogs.

Let's create the template **search.html** that you have already created in the folder templates of your Django app '**blog_app**'.

SEARCH HTML:

Add the following code in the **search.html** file.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Search Blog</title>
</head>
<style>
  table,
  th,
  td
  {
    border: 2px solid;
  }
</style>

<body>
  <h2 style="text-align:center"> Blog Informationn </h2>
  <table align="center" style="margin: 0px auto;">
    <thead>
      <tr>
        <th>Blog ID</th>
        <th>Title</th>
        <th>Author</th>
        <th>Start Date</th>
        <th>End Date</th>
        <th>Edit Blog</th>
```

```
        <th>Remove Blog</th>
    </tr>
</thead>
<tbody>
    {% for blog in blogs %}
    <tr>
        <td>{{blog.blog_id}}</td>
        <td>{{blog.title}}</td>
        <td>{{blog.author_name}}</td>
        <td>{{blog.start_date}}</td>
        <td>{{blog.end_date}}</td>
        <td>
            <a href="/update/{{blog.pk}}">Update</a>
        </td>
        <td>
            <a href="/delete/{{blog.pk}}">Delete</a>
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>
</body>

</html>
```

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta http-equiv="X-UA-Compatible" content="IE=edge">
7     <meta name="viewport" content="width=device-width, initial-scale=1">
8     <title>Search Blog</title>
9 </head>
10 <style>
11     table,
12     th,
13     td
14     {
15         border: 2px solid;
16     }
17 </style>
18
19 <body>
20     <h2 style="text-align:center"> Blog Informationn </h2>
21     <table align="center" style="margin: 0px auto;">
22         <thead>
23             <tr>
24                 <th>Blog ID</th>

```

In the above code, the HTML Template shows the list of all the blogs entered by users in the Table. The following are the headers of the table:

- **Blog ID:** Shows Blog ID.
- **Title:** Shows Title of the Blog.
- **Author:** Shows who writes the specific blog.
- **Start Date:** When the author starts writing this blog.
- **End Date:** When the author ends this blog.
- It also shows the following action buttons:
- **Update:** To edit specific entries added by the users.
- **Delete:** To remove specific entries added by the users.

Blog Informationn						
Blog ID	Title	Author	Start Date	End Date	Edit Blog	Remove Blog

Create the next view which is called **update_blog**, this view takes the id of the specific blog and allows the user to update blog details.

UPDATE VIEW:

Add the following code in the **views.py** file.

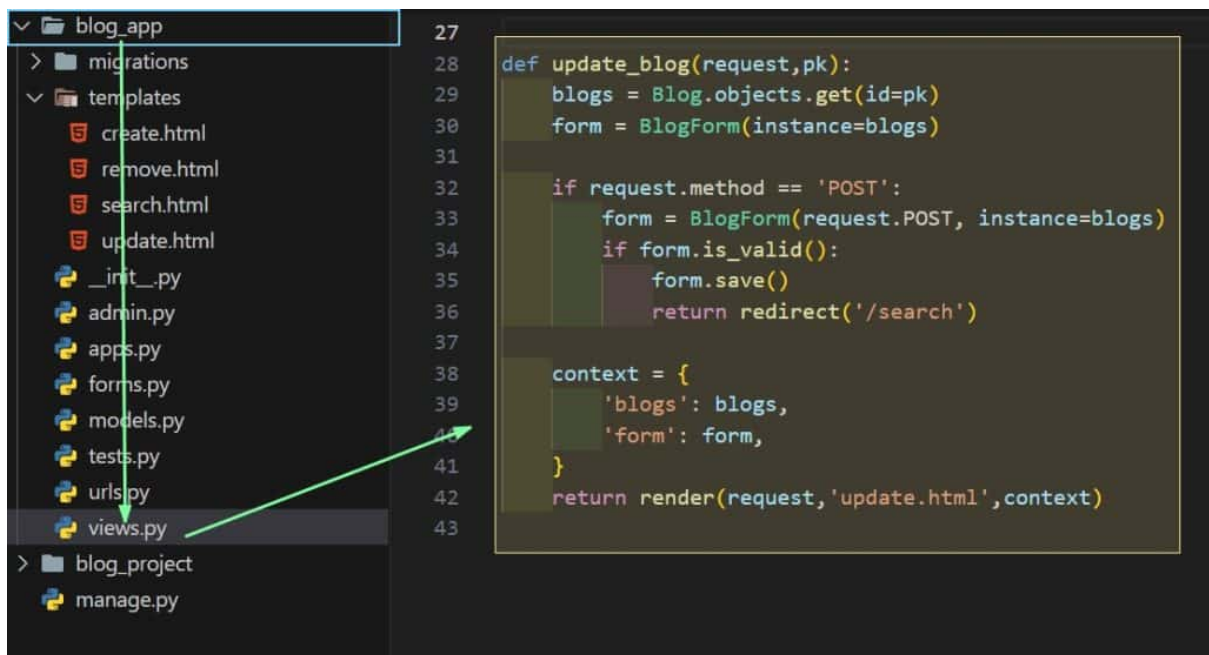
```
from django.shortcuts import render, redirect
```

```
from Blog.forms import BlogForm
```

```
from Blog.models import Blog
```

```
# Update Blog
```

```
def update_blog(request,pk):  
    blogs = Blog.objects.get(id=pk)  
    form = BlogForm(instance=blogs)  
  
    if request.method == 'POST':  
        form = BlogForm(request.POST, instance=blogs)  
        if form.is_valid():  
            form.save()  
            return redirect('/search')  
  
    context = {  
        'blogs': blogs,  
        'form': form,  
    }  
    return render(request,'update.html',context)
```



In the above code, the view `update_blog` is similar to the `create` view that you have created in the above sections.

- The only difference is that here in the `update_blog` function, passing `blogs instance` to the `BlogForm`.
- Then updates are performed based on the **primary key** of the blog or the `pk` representing the id of the blog that you want to update.

Also, create an HTML template that updates a specific field of the blog or the user can specify the details that need to be updated.

UPDATE HTML:

Add the following code in the `update.html` file.

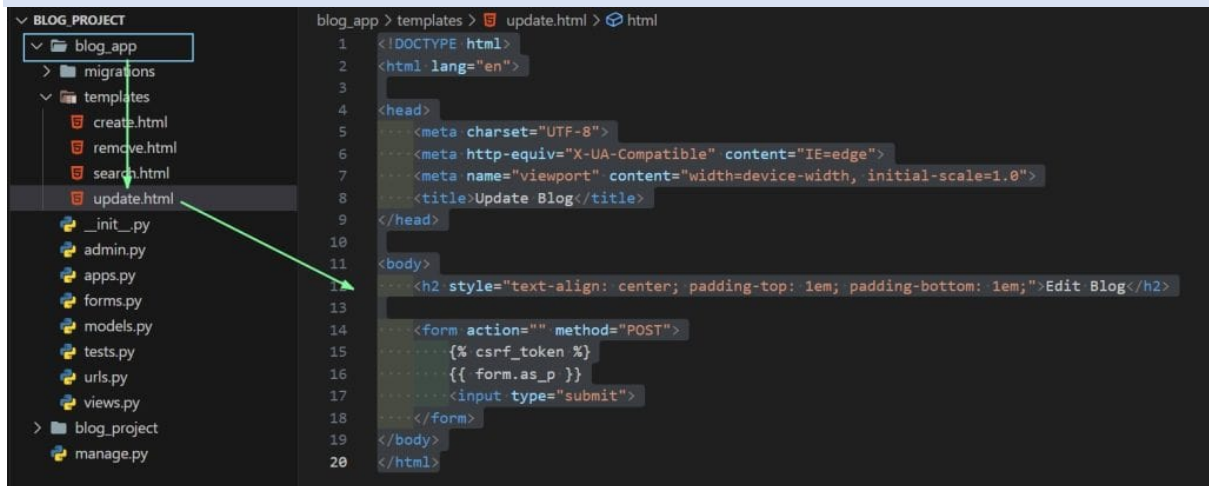
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Update Blog</title>
</head>
```

```

<body>
    <h2 style="text-align: center; padding-top: 1em; padding-bottom: 1em;">Edit
    Blog</h2>

    <form action="" method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <input type="submit">
    </form>
</body>
</html>

```



In the above code, the form tag is used to create an HTML form. By using this form users can edit any field of the specific blog which they want to.

Read: [Python Django vs Pyramid](#)

Finally, create a view function named **delete_blog** that is used to delete the specific blog from the database that is added by the user.

DELETE VIEW:

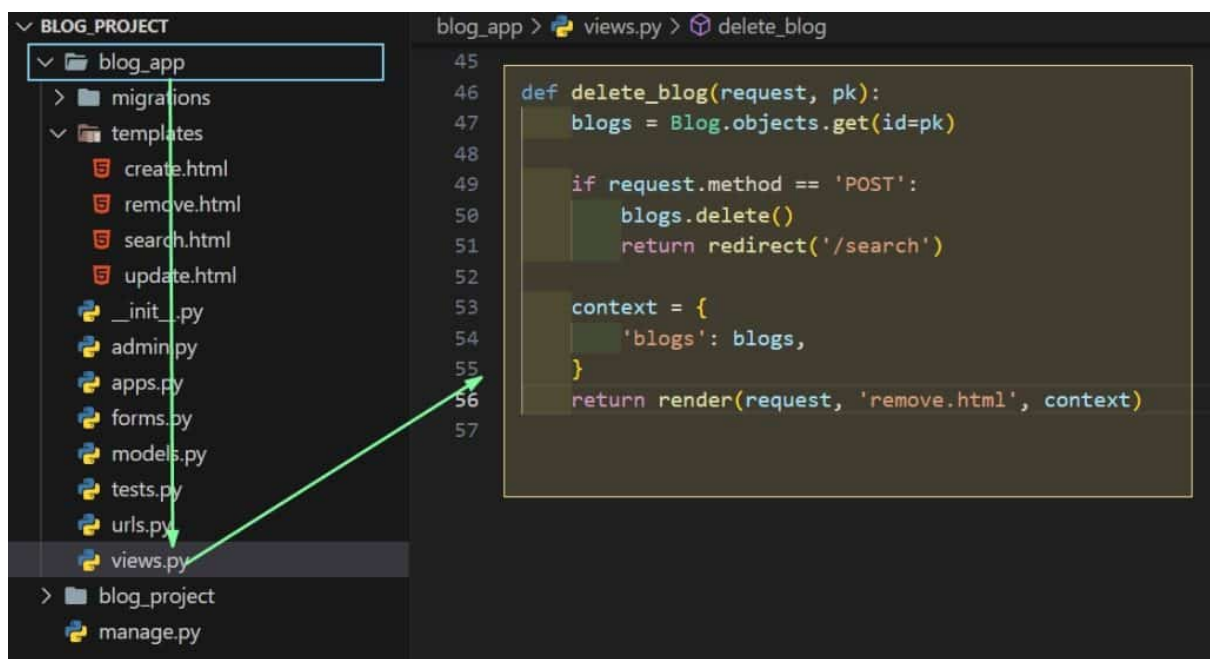
Add the following code in the **views.py** file.

from django.shortcuts import render, redirect

```
from Blog.forms import BlogForm
from Blog.models import Blog
# Delete Blog
def delete_blog(request, pk):
    blogs = Blog.objects.get(id=pk)

    if request.method == 'POST':
        blogs.delete()
        return redirect('/search')

    context = {
        'blogs': blogs,
    }
    return render(request, 'remove.html', context)
```



In the above code, line 46 defines the view `delete_blog` which accepts the two parameters, the `request` or the `pk` (which is the id of the specific blog).

Then at line 47, the blog with **id** which is equal to the **pk** is retrieved from the database using the **Blog.objects.get(id=pk)**. After that, the retrieved blog is deleted by calling the method **delete()** on the blog object in line 50.

Finally, the user is redirected to the page **'search/'**.

Also, check: Simple Contact Form for website in Python Django

Now create an HTML template **remove.html** that allows the user to delete the specific blog.

REMOVE HTML:

Add the following code in the **remove.html** file of the templates folder in your Django app **'blog_app'**.

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Delete Blog</title>
</head>

<body>
    <form action="" method="POST">
        {% csrf_token %}

        <br><br>
        <div class="alert alert-danger" role="alert">
```

```

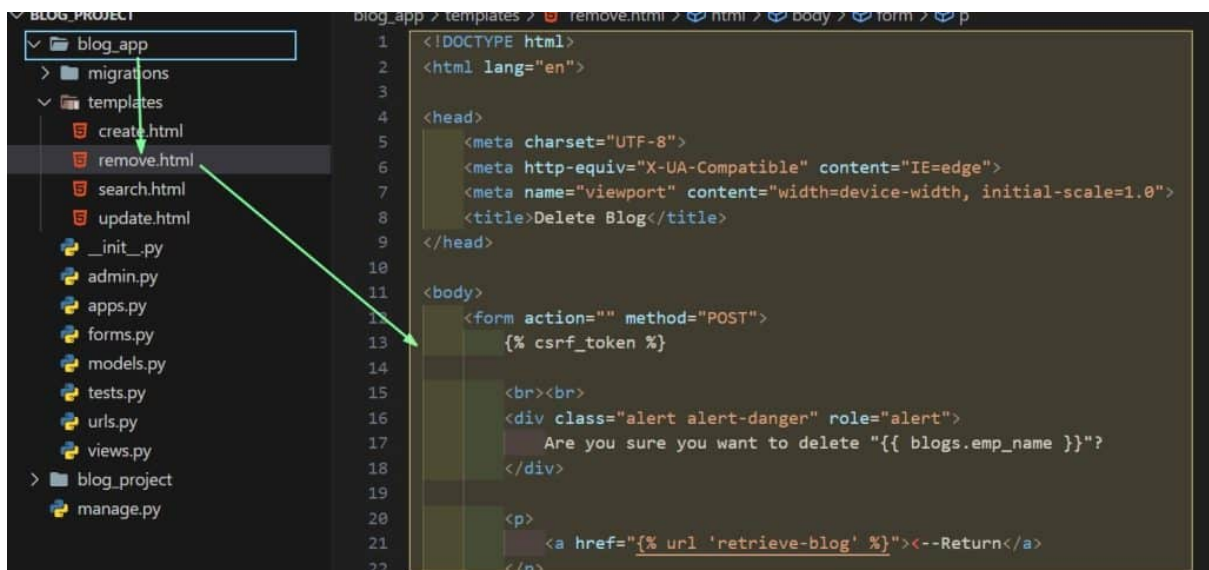
        Are you sure you want to delete "{{ blogs.emp_name }}"?
    </div>

    <p>
        <a href="{% url 'retrieve-blog' %}">--Return</a>
    </p>

    <p>
        <input type="submit" value="Confirm">
    </p>
</form>
</body>

</html>

```



In the above code, create an HTML Template that allows the user to remove the specific blog from the website.

- And It also **confirms** once before the Deletion process.
- Users can also navigate to the previous page i.e. search template by clicking on the **Return** Button.

Python Django MySQL CRUD Migrating and Running Server

Now it is time to run the server and see how to perform CRUD operation. So for that, first migrate the model that comes with Django and the mode that you have created. Remember: Make sure you have created the empty database named **Blog** in your MySQL. if you don't create then you will get the error. After creating the database **Blog**, follow the below steps:

Run the following commands in your terminal one by one.

- *python manage.py migrate*
- *python manage.py makemigrations*
- *python manage.py migrate*

```
• (env) PS D:\Django\django_crud\blog_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
• (env) PS D:\Django\django_crud\blog_project> python manage.py makemigrations
Migrations for 'blog_app':
  blog_app\migrations\0001_initial.py
    - Create model Blog
• (env) PS D:\Django\django_crud\blog_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog_app, contenttypes, sessions
Running migrations:
  Applying blog_app.0001_initial... OK
• (env) PS D:\Django\django_crud\blog_project>
```

Then, we have to run the server, for this type of the following command in the terminal.

- *python manage.py runserver*

Start the server and access the form by entering the URL as <http://127.0.0.1:8000>.

First, enter all the details in the form to create a new blog and click on **Submit Button**.

Create New Blog

BlogId: 100

Title: Introduction to Django

Author Name: Mary Smith

Start Date: 2022-03-04

End Date: 2022-03-06

Submit

Let's see how it looks in MySQL.

☐ Show all

Number of rows: 25

Filter rows: Search this table

+ Options

				id	BlogId	Title	Author_Name	Start_Date	End_Date
☐	Edit	Copy	Delete	8	100	Introduction to Django	Mary Smith	2022-03-04	2022-03-06

When we click on the Submit button, it renders to search HTML template.

Blog Informationn

Blog ID	Title	Author	Start Date	End Date	Edit Blog	Remove Blog
100	Introduction to Django	Mary Smith	March 4, 2022	March 6, 2022	Update	Delete

We can see that if you want to edit a blog, we can use the **Update Button**, and also for deleting a blog, you can use the **Delete Button**.

Now, see how it looks when we click on the **Update** button.

Edit Blog

BlogId:

Title:

Author Name:

Start Date:

End Date:

UPDATE HTML TEMPLATE

Now, we can edit any of the fields.

Edit Blog

BlogId:

Title:

Author Name:

Start Date:

End Date:

- Here I edit the End Date field of the form.
- I changed the End Date from 2022-03-06 to 2022-03-08.

Now, see what will happen when we click on the **Submit** button.

Blog Information

Blog ID	Title	Author	Start Date	End Date	Edit Blog	Remove Blog
100	Introduction to Django	Mary Smith	March 4, 2022	March 8, 2022	Update	Delete

Here, we see that the End Date has been updated.

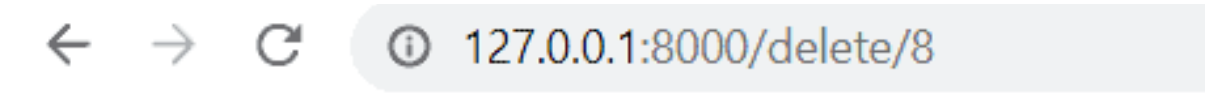
☐ Show all	Number of rows: 25	Filter rows: Search this table
-----------------------------------	--------------------	---

+ Options

	id	BlogId	Title	Author_Name	Start_Date	End_Date
☐ Edit Copy Delete	8	100	Introduction to Django	Mary Smith	2022-03-04	2022-03-08

We can see it also updates blog information at MySQL.

Now, see how it looks when we click on the **Delete** button.



Are you sure you want to delete ""?

[<-- Return](#)

Confirm

Now, you can see that before deletion it confirms once “Are you sure you want to delete”.

Now, see what will happen if we click on **Return**.

Blog Information

Blog ID	Title	Author	Start Date	End Date	Edit Blog	Remove Blog
100	Introduction to Django	Mary Smith	March 4, 2022	March 8, 2022	Update	Delete

We can see that it takes us back to blog information.

Now, see what will happen if we click on **Confirm**.

Blog Information

Blog ID	Title	Author	Start Date	End Date	Edit Blog	Remove Blog
---------	-------	--------	------------	----------	-----------	-------------

We can see that it deletes the blog.

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

id	BlogId	Title	Author_Name	Start_Date	End_Date
----	--------	-------	-------------	------------	----------

Query results operations

 [Create view](#)

We can see it will also delete blogs from the MySQL database.

In this Python Django tutorial, you implemented the most fundamental operations which are called **CRUD** by building the blog-based project. These operations allow users to create, read, update and delete the specific data.

You may also like to read:

- [Get URL parameters in Django](#)
- [How to create model in Django](#)
- [Python Django form validation](#)

Reference

<https://pythonguides.com/python-django-mysql-crud/>